

CHALLENGE WTC 2025 · FIAP · ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

WTC Chat App

Plataforma de comunicação CRM — app iOS nativo + backend Java robusto, integrados via APIs REST reais e mensageria em tempo real.

SPRINT 2 Backend real · sem mocks

Entrega: 19/05 · 23h00 | Equipe 2TDS

INTEGRANTES

Equipe

Arthur Cavalcanti Granja

RM 560650

Análise e Desenvolvimento de Sistemas — 2TDS · FIAP
Challenge WTC 2025 · Sprint 2 — Backend + Integração

Visão Geral

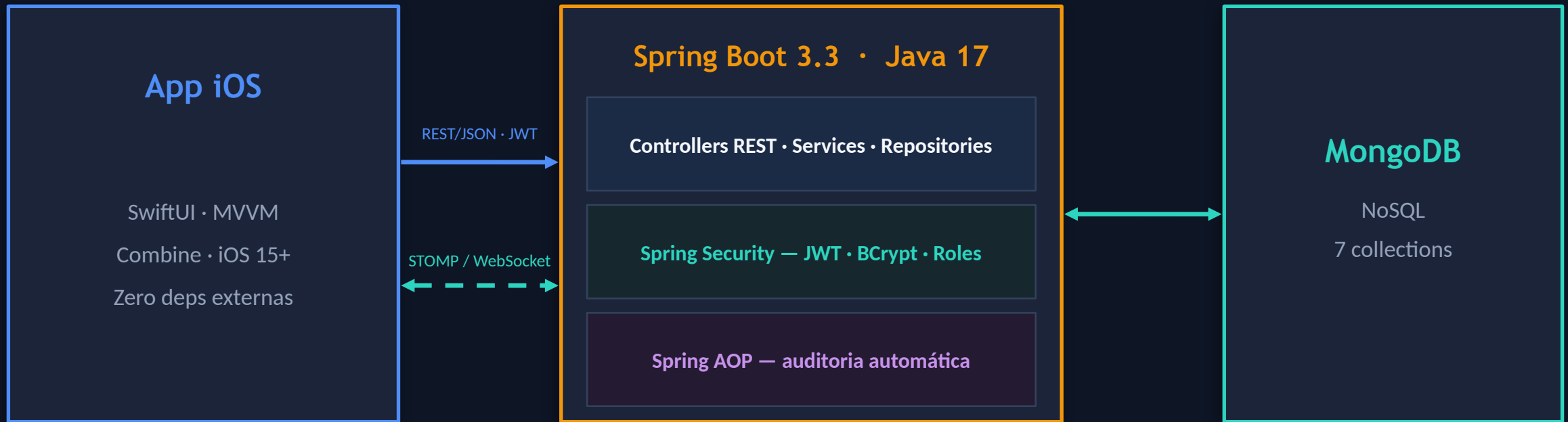
PROBLEMA

- Comunicação fragmentada entre empresa e clientes
- Falta de personalização nas mensagens de marketing
- Notificações sem interatividade
- Dificuldade em segmentar o público-alvo

SOLUÇÃO

- Mensagens segmentadas 1:1 e por grupo/tag
- Push + pop-up in-app, histórico de chat
- Botões de ação e deeplinks interativos
- CRM no app (visão operador) + tempo real (WebSocket)

Arquitetura do Backend



Fluxo de envio: Controller → Service → Repository → MongoDB; em paralelo cria Notification e faz broadcast via WebSocketService para /topic/messages/{userId}. AuditAspect (AOP) registra toda operação de escrita.

Stack & Segurança

iOS	Swift 5.9 · SwiftUI · Combine · MVVM · iOS 15+ · sem dependências externas (URLSession + STOMP manual)
Backend	Java 17 · Spring Boot 3.3 · Spring Security · Spring Data MongoDB · Spring WebSocket · Spring AOP · Lombok · JWT
Banco	MongoDB 6.0+ — NoSQL orientado a documentos, 7 collections
Auth	JWT (access 24h / refresh 7d) · BCrypt · roles OPERATOR / CLIENT
Realtime	WebSocket STOMP — /topic/messages/{userId} · /topic/notifications/{userId}
Governança	Auditoria automática via AOP (@AfterReturning) em toda operação de escrita

Modelo de Dados

users	id · email* · password · fullName · phone · role · tags[] · status
customers	id · userId* · tags[] · score · status · notes[] · segmentIds[]
segments	id · name · description · tags[] · createdBy
messages	id · type · senderId · recipientId* · segmentTags[] · content · status · readAt · starred
campaigns	id · name · segmentId · content · deeplink · status · sentBy · messageCount
notifications	id · userId* · title · body · type · read · messageId
audit_logs	id · userId · action · resource · resourceId · details · ipAddress · timestamp

* campo indexado · content = { title, body, imageUrl, buttons[] } · Inbox: \$or [recipientId = userId, segmentTags ∈ userTags]

Endpoints

Auth • CRM • Segmentos

Método	Rota	Acesso	Descrição
POST	/api/auth/login	público	Login → JWT + perfil
POST	/api/auth/register	público	Registrar usuário
POST	/api/auth/refresh	público	Renovar token
GET	/api/customers	OPERATOR	Listar (filtros tag/status/minScore)
POST	/api/customers	OPERATOR	Criar cliente CRM
GET/PUT	/api/customers/{id}	OPERATOR	Detalhe / atualizar
GET	/api/customers/{id}/timeline	OPERATOR	Perfil 360° (msgs + notas)
POST	/api/customers/{id}/notes	OPERATOR	Anotação rápida
GET/POST	/api/segments	OPERATOR	Listar / criar segmento
PUT/DELETE	/api/segments/{id}	OPERATOR	Atualizar / remover

Endpoints

Mensagens · Campanhas · Notificações · Auditoria

Método	Rota	Acesso	Descrição
POST	/api/messages	autenticado	Enviar msg (1:1 ou segmento)
GET	/api/messages/{id}	autenticado	Detalhe da mensagem
GET	/api/inbox/{customerId}	autenticado	Inbox do cliente
GET	/api/messages/sent/{senderId}	autenticado	Histórico de enviadas
PUT	/api/messages/{id}/read	autenticado	Marcar como lida
PUT	/api/messages/{id}/star	autenticado	Favoritar
GET/POST	/api/campaigns	OPERATOR	Listar / criar campanha
POST	/api/campaigns/{id}/send	OPERATOR	Enviar para segmento
GET/PUT	/api/notifications	autenticado	Listar / marcar lida(s)
GET	/api/audit-logs	OPERATOR	Logs (filtros resource/userId)

Modelo Flexível de Mensagem

REQUEST • POST /api/messages

```
{
  "type": "CHAT",
  "recipient_id": "uuid-cliente",
  "content": {
    "title": "Campanha Especial",
    "body": "Participe do evento!",
    "image_url": "https://.../img.jpg",
    "buttons": [
      { "label": "Inscrever-se",
        "action": "https://wtc.com/x" },
      { "label": "Ver perfil",
        "action": "deeplink://profile" }
    ]
  }
}
```

RESPONSE • 201 Created

```
{
  "id": "uuid-msg",
  "type": "CHAT",
  "sender_id": "uuid-operador",
  "recipient_id": "uuid-cliente",
  "content": { ... },
  "status": "SENT",
  "starred": false,
  "created_at": "2026-05-19T23:00:00Z"
}

status: SENT → DELIVERED
        → READ → FAILED
```

Diferenciais

Tempo real

WebSocket/STOMP: nova mensagem aparece instantaneamente no app, sem polling.

Governança (AOP)

Auditoria automática de toda escrita via Spring AOP — zero código manual.

Multi-role

Roteamento por papel: CLIENT (inbox) vs OPERATOR (CRM completo no app).

Botões interativos

deeplink:// (navegação interna), https:// (externo), copy: (clipboard).

Status de mensagem

Ciclo SENT → DELIVERED → READ → FAILED rastreado no documento.

Zero deps no iOS

App nativo sem libs externas — STOMP client implementado manualmente.

Como rodar o projeto

Backend

```
brew install openjdk@17 \
  mongodb-community
brew services start \
  mongodb-community

cd backend
JAVA_HOME=$(brew --prefix \
  openjdk@17) ./mvnw \
  spring-boot:run
# http://localhost:8080
```

App iOS

```
brew install xcodegen
cd Challenge-WTC
xcodegen generate
open WTCChatApp.xcodeproj
# Cmd+R (iPhone 15+)
```

Entregáveis: backend.zip ·
app.zip + WTCChatApp.ipa ·
apresentação (.pptx/.pdf)